

Fixed Points, Basins, and the Price of Information: Measured Laws of Recursive Reasoning Models

Aakash Marthandan
aakashmarthandan@gmail.com

Abstract

Recursive reasoning models solve Sudoku-Extreme by applying a small network to a carried state, step after step, until the grid stops changing. They reach high accuracy, but why the loop works has not been measured, and their reported numbers depend on conventions that no paper states (how many steps are run, whether the weights are averaged, which checkpoint is selected) by more than the differences between the models. We treat the loop as a flow toward a fixed point and build a set of instruments that measure it directly: whether a trained model commits to cells and propagates from them or averages over possibilities, from which starting states it reaches the solution, which wrong grids it converges to, and what its confident commitments cost. Applied to the publicly released models and to forty grids trained one design choice at a time and evaluated on identical puzzles, the instruments give seven laws. The loop, not the parameter count or the operator, decides whether a model commits or averages; within the class, the models that decide least in the first step and revise most finish highest at depth. Every model that commits reports full confidence on the puzzles it fails, where half of its committed cells are wrong; a second readout trained on the carried state recovers which puzzles are failing, and no readout recovers which cells. Selecting an answer by its convergence residual works without a verifier exactly when training has exposed the model to its own generated states and removed the wrong fixed points. Every training recipe eventually memorizes the thousand training puzzles, at a rate set by the cell's capacity rather than by the augmentation, so the choice of checkpoint is part of the result. Doubling the number of steps raises accuracy every time and never unsolves a solved puzzle. Exact symmetry of the state does the work that relabeled training data does for other models, and insensitivity to the starting state can be trained in. The laws say what to build. The Decimating Equilibrium Cell keeps the block and two-timescale loop that the laws identify as the source of commitment and places it on a nine-field state with every parameter shared across the digits, so that relabeling the digits relabels the output exactly. Under the thousand-puzzle training convention and with no digit augmentation, three training seeds of the cell reach $95.0 \pm 0.7\%$ single-pass exact accuracy at 16 steps on the full 422,786-puzzle test set, and a narrower cell of 789 thousand parameters reaches 95.9 % at 16 steps and 99.2 % at 64 steps on the same test set, against 86.5 % and 93.2 % for the best released checkpoint of the same convention evaluated with the same code on the same puzzles; the narrower cell costs 1.15 times that checkpoint's arithmetic per step at 16 % of its parameters, the wider one 3.9 times at 55 %. With 128 random restarts selected by the residual and no verifier, the narrower cell reaches 99.9 % on a 5,000-puzzle subsample, beside the 99.8 % that checkpoint reports with 128 restarts; solvers trained on the full split of millions of puzzles report up to 99.9 % in a single stochastic pass. Every number is placed in a table whose columns are the training set, the number of steps, the number of restarts, the selection rule and the arithmetic per puzzle, and every result was predicted in writing before its data and scored afterward, misses included. The same instruments read the Abstraction and Reasoning Corpus, which these models already solve in substantial part, as a different regime: there the answer cannot be verified, what limits a model is which rules it holds a basin for rather than how far it can search, and a failure is a confident wrong fixed point rather than an unfinished propagation. The two benchmarks that the literature treats as one family are therefore a verification game and a generalization task, and we state which instruments and mechanisms carry over between them and what the generalization task asks of the next model. Together, the results say what a recursive reasoner commits to, from which starting states it can be reached, and what its accuracy costs.

1 Introduction

[**TODO:** Introduction.]

2 Setup: the cells, the loop, the protocol

[**TODO:** Setup: the cells, the loop, the protocol.]

3 The instrument suite

[**TODO:** The instrument suite.]

4 Results on Sudoku-Extreme

[**TODO:** Results on Sudoku-Extreme.]

5 The field's models through the same instruments

[**TODO:** The field's models through the same instruments.]

6 Two problems in one boat: Sudoku and ARC through the same instruments

[**TODO:** The field evaluates Sudoku-Extreme and ARC as one benchmark family; the same instruments separate a verification game from a generalization task. The ARC-era results in retrospect (retention, basin existence vs reachability, the reachability wall, the priced-code laws, the init-distribution law), then the measured differences as a bold run-in ledger, the portability table, and the hand-off to the ARC paper.]

7 Related work

[**TODO:** A narrative of results (HRM, TRM, EqR, SE-RRM, PTRM, CGAR, CMM, GRAM, guided reasoning; the classical decoders) ending at the gap: none measures the flow; the parity numbers are protocol conventions; the two benchmarks are evaluated as one family.]

8 Limitations and future directions

[**TODO:** The honest ledger in the bold run-in form: the compute column; one verification domain at frontier accuracy and one generalization domain at a measured, sub-frontier accuracy; the calibrated-commitment gap measured, not closed; the selection instrument; the ARC scale-matching gap; what would settle each.]

9 Conclusion

[**TODO:** The one thesis sentence, then the one suggestive result.]

References

A Appendix

[**TODO:** Appendix.]